

```
# =====  
#           RFM ANALYSIS & CUSTOMER SEGMENTATION  
# =====  
  
# -----  
# 1. IMPORT REQUIRED LIBRARIES  
# -----  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# -----  
# 2. LOAD CUSTOMER-LEVEL RFM BASE DATA  
# -----  
  
# Load the RFM base file created in Phase 2  
rfm = pd.read_csv("customer_rfm_base.csv")  
  
# Display first few records  
print("First 5 records:")  
display(rfm.head())  
  
# Check dataset shape  
print("Shape:", rfm.shape)
```

```
# Check column names
print("\nColumns:")
print(rfm.columns.tolist())
```

```
# -----
# 3. BASIC DATA CHECK
# -----
```

```
# Check data types
print("\nData Types:")
print(rfm.dtypes)
```

```
# Check missing values
print("\nMissing Values:")
print(rfm.isnull().sum())
```

```
# Check duplicate customers
print("\nDuplicate Customer IDs:", rfm["customer_id"].duplicated().sum())
```

```
# -----  
# 4. CHECK RFM STATISTICS  
# -----  
  
# Display descriptive statistics  
print("\nRFM Statistics:")  
display(  
    rfm[  
        ["recency", "frequency", "monetary"]  
    ].describe()  
)  
  
# -----  
# 5. CHECK RFM DISTRIBUTIONS  
# -----  
  
# Create separate plots for Recency, Frequency and Monetary  
  
plt.figure(figsize=(8, 5))  
sns.histplot(rfm["recency"], bins=30, kde=True)  
plt.title("Recency Distribution")  
plt.xlabel("Recency (Days)")  
plt.ylabel("Number of Customers")  
plt.show()
```

```
plt.figure(figsize=(8, 5))
sns.histplot(rfm["frequency"], bins=30, kde=True)
plt.title("Frequency Distribution")
plt.xlabel("Number of Purchases")
plt.ylabel("Number of Customers")
plt.show()
```

```
plt.figure(figsize=(8, 5))
sns.histplot(rfm["monetary"], bins=30, kde=True)
plt.title("Monetary Distribution")
plt.xlabel("Total Spending")
plt.ylabel("Number of Customers")
plt.show()
```

```
# -----
# 6. CREATE RFM SCORES
# -----
```

```
# RFM scoring:
```

```
#
```

```
# Recency:
```

```
# Lower recency = Better customer
```

```
# Recent customers receive higher scores
```

```
#
```

```
# Frequency:
```

```
# Higher frequency = Better customer
```

```
# Monetary:  
# Higher spending = Better customer
```

```
# -----  
# 6A. FUNCTION FOR QUANTILE-BASED SCORING  
# -----
```

```
def create_quantile_score(series, reverse=False):  
    """  
    Convert a numerical column into a score from 1 to 5  
    using quantiles.  
  
    reverse=True:  
        Lower values receive higher scores.  
        Used for Recency.  
  
    reverse=False:  
        Higher values receive higher scores.  
        Used for Frequency and Monetary.  
    """  
  
    # Rank values first to handle large numbers of tied values  
    ranked_series = series.rank(method="first")
```

```
# Create five quantile groups
scores = pd.qcut(
    ranked_series,
    q=5,
    labels=False
) + 1
```

```
# Reverse the score for Recency
if reverse:
    scores = 6 - scores

return scores.astype(int)
```

```
# -----
# 6B. CALCULATE RFM SCORES
# -----
```

```
# Recency score
# Recent customers = high score
rfm["R_Score"] = create_quantile_score(
    rfm["recency"],
    reverse=True
)
```

```
# Frequency score
# Frequent customers = high score
rfm["F_Score"] = create_quantile_score(
    rfm["frequency"],
    reverse=False
)
```

```
# Monetary score
# High-spending customers = high score
rfm["M_Score"] = create_quantile_score(
    rfm["monetary"],
    reverse=False
)
```

```
# -----
# 7. CREATE COMBINED RFM SCORE
# -----
```

```
# Combine the three individual scores
rfm["RFM_Score"] = (
    rfm["R_Score"] +
    rfm["F_Score"] +
    rfm["M_Score"]
)
```

```
# Create a combined score such as 555, 453, 215 etc.
```

```
rfm["RFM_Score_Code"] = (  
    rfm["R_Score"].astype(str) +  
    rfm["F_Score"].astype(str) +  
    rfm["M_Score"].astype(str)  
)
```

```
# -----
```

```
# 8. DISPLAY RFM SCORES
```

```
# -----
```

```
display(  
    rfm[  
        [  
            "customer_id",  
            "recency",  
            "frequency",  
            "monetary",  
            "R_Score",  
            "F_Score",  
            "M_Score",  
            "RFM_Score",  
            "RFM_Score_Code"  
        ]  
    ].head(20)  
)
```

```
# -----  
# 9. CHECK SCORE DISTRIBUTIONS  
# -----  
  
print("\nR Score Distribution:")  
print(rfm["R_Score"].value_counts().sort_index())  
  
print("\nF Score Distribution:")  
print(rfm["F_Score"].value_counts().sort_index())  
  
print("\nM Score Distribution:")  
print(rfm["M_Score"].value_counts().sort_index())  
  
# -----  
# 10. CREATE BUSINESS-ORIENTED CUSTOMER SEGMENTS  
# -----  
  
def assign_rfm_segment(row):  
    """  
    Assign a business-friendly customer segment  
    based on RFM scores.  
    """  
  
    R = row["R_Score"]  
    F = row["F_Score"]  
    M = row["M_Score"]
```

```
# -----  
# Champions  
# -----  
if R >= 4 and F >= 4 and M >= 4:  
    return "Champions"  
  
# -----  
# Loyal Customers  
# -----  
elif R >= 3 and F >= 4 and M >= 3:  
    return "Loyal Customers"  
  
# -----  
# Potential Loyalists  
# -----  
elif R >= 4 and F >= 2 and M >= 2:  
    return "Potential Loyalists"  
  
# -----  
# New Customers  
# -----  
elif R >= 4 and F <= 2:  
    return "New Customers"
```

```
# -----  
# At Risk  
# -----  
elif R <= 2 and F >= 3 and M >= 3:  
    return "At Risk"  
  
# -----  
# Cannot Lose Them  
# -----  
elif R <= 2 and F >= 4 and M >= 4:  
    return "Cannot Lose Them"  
  
# -----  
# Need Attention  
# -----  
elif R <= 3 and F <= 3 and M >= 3:  
    return "Need Attention"  
  
# -----  
# Hibernating  
# -----  
elif R <= 2 and F <= 2:  
    return "Hibernating"
```

```
# -----  
# Other Customers  
# -----  
else:  
    return "Others"
```

```
# Apply segmentation  
rfm["Customer_Segment"] = rfm.apply(  
    assign_rfm_segment,  
    axis=1  
)
```

```
# -----  
# 11. CHECK CUSTOMER SEGMENT COUNTS  
# -----
```

```
segment_counts = (  
    rfm["Customer_Segment"]  
    .value_counts()  
    .reset_index()  
)
```

```
segment_counts.columns = [  
    "Customer_Segment",
```

```
    "Customer_Count"  
]  
  
print("\nCustomer Segment Counts:")  
display(segment_counts)  
  
# -----  
# 12. CALCULATE SEGMENT PERCENTAGE  
# -----  
  
total_customers = len(rfm)  
  
segment_counts["Customer_Percentage"] = (  
    segment_counts["Customer_Count"]  
    / total_customers  
    * 100  
)  
  
print("\nCustomer Segment Distribution:")  
display(segment_counts)
```

```
# -----  
# 13. CREATE SEGMENT SUMMARY  
# -----
```

```
segment_summary = (  
    rfm  
    .groupby("Customer_Segment")  
    .agg(  
        Customer_Count=("customer_id", "count"),  
        Total_Revenue=("monetary", "sum"),  
        Average_Spend=("monetary", "mean"),  
        Average_Frequency=("frequency", "mean"),  
        Average_Recency=("recency", "mean"),  
        Average_RFM_Score=("RFM_Score", "mean"),  
        Churn_Rate=("churn", "mean")  
    )  
    .reset_index()  
)
```

```
# Convert churn into percentage  
segment_summary["Churn_Rate"] = (  
    segment_summary["Churn_Rate"] * 100  
)
```

```
# Calculate customer percentage
segment_summary["Customer_Percentage"] = (
    segment_summary["Customer_Count"]
    / total_customers
    * 100
)

# Round numerical columns
segment_summary = segment_summary.round(2)

# Sort by total revenue
segment_summary = segment_summary.sort_values(
    "Total_Revenue",
    ascending=False
)

print("\nRFM Segment Summary:")
display(segment_summary)
```

```
# -----  
# 14. VISUALIZE CUSTOMER SEGMENT DISTRIBUTION  
# -----  
  
plt.figure(figsize=(6, 3))  
  
segment_plot = (  
    rfm["Customer_Segment"]  
    .value_counts()  
    .sort_values(ascending=False)  
)  
  
sns.barplot(  
    x=segment_plot.values,  
    y=segment_plot.index  
)  
  
plt.title("Customer Segment Distribution")  
plt.xlabel("Number of Customers")  
plt.ylabel("Customer Segment")  
plt.tight_layout()  
plt.show()
```

```
# -----  
# 15. VISUALIZE REVENUE BY CUSTOMER SEGMENT  
# -----  
  
revenue_plot = (  
    rfm  
    .groupby("Customer_Segment")["monetary"]  
    .sum()  
    .sort_values(ascending=False)  
)  
  
plt.figure(figsize=(6,3))  
  
sns.barplot(  
    x=revenue_plot.values,  
    y=revenue_plot.index  
)  
  
plt.title("Total Revenue by Customer Segment")  
plt.xlabel("Total Revenue")  
plt.ylabel("Customer Segment")  
plt.tight_layout()  
plt.show()
```

```
# -----  
# 16. VISUALIZE AVERAGE CUSTOMER SPENDING  
# -----
```

```
avg_spend_plot = (  
    rfm  
    .groupby("Customer_Segment")["monetary"]  
    .mean()  
    .sort_values(ascending=False)  
)  
  
plt.figure(figsize=(6,3))  
  
sns.barplot(  
    x=avg_spend_plot.values,  
    y=avg_spend_plot.index  
)  
  
plt.title("Average Customer Spending by Segment")  
plt.xlabel("Average Spending")  
plt.ylabel("Customer Segment")  
plt.tight_layout()  
plt.show()
```

```
# -----  
# 17. VISUALIZE AVERAGE RECENCY  
# -----
```

```
avg_recency_plot = (  
    rfm  
    .groupby("Customer_Segment")["recency"]  
    .mean()  
    .sort_values(ascending=True)  
)  
  
plt.figure(figsize=(6,3))  
  
sns.barplot(  
    x=avg_recency_plot.values,  
    y=avg_recency_plot.index  
)  
  
plt.title("Average Recency by Customer Segment")  
plt.xlabel("Average Recency (Days)")  
plt.ylabel("Customer Segment")  
plt.tight_layout()  
plt.show()
```

```
# -----  
# 18. VISUALIZE AVERAGE PURCHASE FREQUENCY  
# -----  
  
avg_frequency_plot = (  
    rfm  
    .groupby("Customer_Segment")["frequency"]  
    .mean()  
    .sort_values(ascending=False)  
)  
  
plt.figure(figsize=(6,3))  
  
sns.barplot(  
    x=avg_frequency_plot.values,  
    y=avg_frequency_plot.index  
)  
  
plt.title("Average Purchase Frequency by Segment")  
plt.xlabel("Average Purchase Frequency")  
plt.ylabel("Customer Segment")  
plt.tight_layout()  
plt.show()
```

```
# -----  
# 19. RFM SCORE HEATMAP  
# -----  
  
rfm_heatmap = (  
    rfm  
    .groupby(  
        ["R_Score", "F_Score"]  
    )["M_Score"]  
    .mean()  
    .unstack()  
)  
  
plt.figure(figsize=(6,4))  
  
sns.heatmap(  
    rfm_heatmap,  
    annot=True,  
    fmt=".2f"  
)  
  
plt.title("Average Monetary Score by Recency and Frequency")  
plt.xlabel("Frequency Score")  
plt.ylabel("Recency Score")  
plt.tight_layout()  
plt.show()
```

```
# 20. ANALYZE CHURN BY RFM SEGMENT
```

```
# -----
```

```
churn_by_segment = (  
    rfm  
    .groupby("Customer_Segment")  
    .agg(  
        Customers=("customer_id", "count"),  
        Churned_Customers=("churn", "sum"),  
        Churn_Rate=("churn", "mean")  
    )  
    .reset_index()  
)
```

```
# Convert churn rate to percentage
```

```
churn_by_segment["Churn_Rate"] = (  
    churn_by_segment["Churn_Rate"] * 100  
)
```

```
churn_by_segment = churn_by_segment.round(2)
```

```
# Sort by churn rate
```

```
churn_by_segment = churn_by_segment.sort_values(  
    "Churn_Rate",  
    ascending=False  
)
```

```
print("\nChurn by Customer Segment:")  
display(churn_by_segment)
```

```
# -----  
# 21. VISUALIZE CHURN RATE BY SEGMENT  
# -----
```

```
plt.figure(figsize=(6,3))
```

```
sns.barplot(  
    data=churn_by_segment,  
    x="Churn_Rate",  
    y="Customer_Segment"  
)
```

```
plt.title("Churn Rate by Customer Segment")  
plt.xlabel("Churn Rate (%)")  
plt.ylabel("Customer Segment")  
plt.tight_layout()  
plt.show()
```

```
# 22. IDENTIFY HIGH-VALUE CUSTOMERS
```

```
# -----
```

```
# Customers with high RFM scores
```

```
high_value_customers = rfm[  
    rfm["RFM_Score"] >= 12  
].copy()
```

```
print(  
    "Number of High-Value Customers:",  
    len(high_value_customers)  
)
```

```
display(  
    high_value_customers[  
        [  
            "customer_id",  
            "customer_name",  
            "recency",  
            "frequency",  
            "monetary",  
            "RFM_Score",  
            "Customer_Segment",  
            "churn"  
        ]  
    ]  
    .sort_values("RFM_Score", ascending=False)  
    .head(20)  
)
```

```
# -----  
# 23. IDENTIFY AT-RISK CUSTOMERS  
# -----  
  
at_risk_customers = rfm[  
    rfm["Customer_Segment"].isin(  
        ["At Risk", "Cannot Lose Them"]  
    )  
].copy()  
  
print(  
    "Number of At-Risk / Cannot-Lose Customers:",  
    len(at_risk_customers)  
)  
  
display(  
    at_risk_customers[  
        [  
            "customer_id",  
            "customer_name",  
            "recency",  
            "frequency",  
            "monetary",  
            "RFM_Score",  
            "Customer_Segment",  
            "churn"  
        ]  
    ]  
    .sort_values("monetary", ascending=False)
```

```
.head(20)
```

```
)
```

```
# -----
```

```
# 24. IDENTIFY HIGH-VALUE CHURNED CUSTOMERS
```

```
# -----
```

```
high_value_churned = rfm[  
    (rfm["churn"] == 1) &  
    (rfm["RFM_Score"] >= 10)  
].copy()
```

```
print(  
    "High-Value Churned Customers:",  
    len(high_value_churned)  
)
```

```
display(  
    high_value_churned[  
        [  
            "customer_id",  
            "customer_name",  
            "recency",  
            "frequency",  
            "monetary",  
            "RFM_Score",  
            "Customer_Segment"  
        ]  
    ]
```

```
]
.sort_values("monetary", ascending=False)
.head(20)
```

```
)
```

```
# -----
# 25. SAVE COMPLETE RFM SEGMENTATION DATASET
# -----
```

```
rfm.to_csv(
    "customer_rfm_segments.csv",
    index=False
```

```
)
```

```
print(
    "Saved: customer_rfm_segments.csv"
```

```
)
```

```
# -----  
# 26. SAVE SEGMENT SUMMARY  
# -----
```

```
segment_summary.to_csv(  
    "rfm_segment_summary.csv",  
    index=False  
)  
  
print(  
    "Saved: rfm_segment_summary.csv"  
)
```

```
# -----  
# 27. SAVE HIGH-VALUE CUSTOMERS  
# -----
```

```
high_value_customers.to_csv(  
    "high_value_customers.csv",  
    index=False  
)  
  
print(  
    "Saved: high_value_customers.csv"  
)
```

```
# -----  
# 28. SAVE AT-RISK CUSTOMERS  
# -----
```

```
at_risk_customers.to_csv(  
    "at_risk_customers.csv",  
    index=False  
)  
  
print(  
    "Saved: at_risk_customers.csv"  
)
```

```
# -----  
# 29. SAVE HIGH-VALUE CHURNED CUSTOMERS  
# -----
```

```
high_value_churned.to_csv(  
    "high_value_churned_customers.csv",  
    index=False  
)  
  
print(  
    "Saved: high_value_churned_customers.csv"  
)
```

```
# -----  
# 30. FINAL STEP 3 CHECK  
# -----  
  
print("\n" + "=" * 60)  
print("STEP 3 COMPLETED")  
print("=" * 60)  
  
print("\nTotal Customers:", len(rfm))  
  
print(  
    "Average Recency:",  
    round(rfm["recency"].mean(), 2),  
    "days"  
)  
  
print(  
    "Average Frequency:",  
    round(rfm["frequency"].mean(), 2)  
)  
  
print(  
    "Average Monetary Value:",  
    round(rfm["monetary"].mean(), 2)  
)  
  
print(  
    "Average RFM Score:",  
    round(rfm["RFM_Score"].mean(), 2)
```

```

print("\nCustomer Segments:")
print(
    rfm["Customer_Segment"]
    .value_counts()
)

print("\nOutput files created:")
print("1. customer_rfm_segments.csv")
print("2. rfm_segment_summary.csv")
print("3. high_value_customers.csv")
print("4. at_risk_customers.csv")
print("5. high_value_churned_customers.csv")

```

First 5 records:

	customer_id	customer_name	recency	frequency	monetary	churn
0	1	Nicole Johnson	57	1	3491	0
1	2	Marie Wright	298	3	7988	0
2	3	Julie Wolfe	88	8	22587	0
3	4	Tracey Smith	126	4	8715	1
4	5	Nancy Jones	170	8	12524	0

Shape: (49673, 6)

Columns:

```
['customer_id', 'customer_name', 'recency', 'frequency', 'monetary', 'churn']
```

Data Types:

```
customer_id      int64
```

```
customer_name    object
```

```
recency          int64
```

```
frequency        int64
```

```
monetary         int64
```

```
churn            int64
```

```
dtype: object
```

Missing Values:

```
customer_id      0
```

```
customer_name    0
```

```
recency          0
```

```
frequency        0
```

```
monetary         0
```

```
churn            0
```

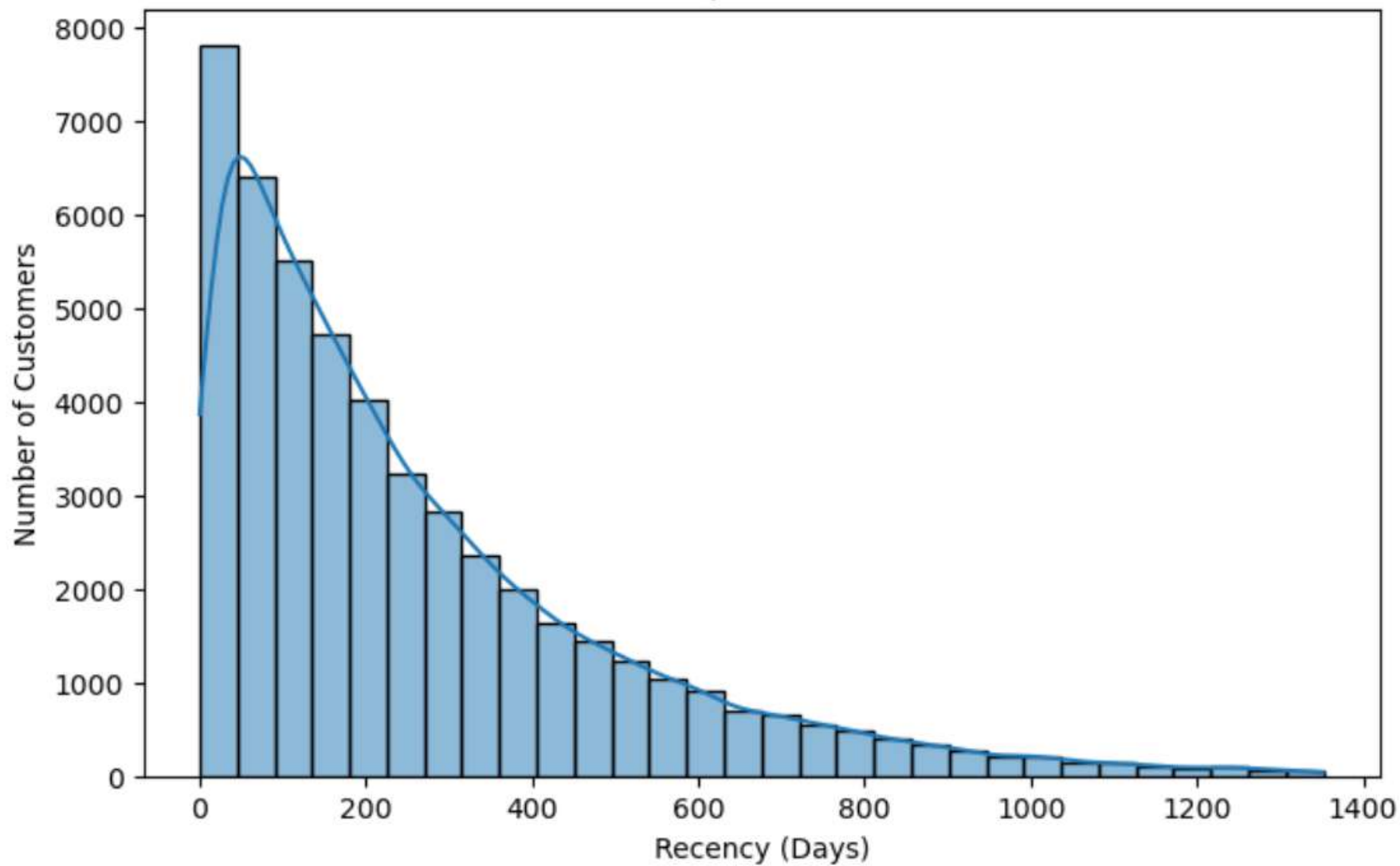
```
dtype: int64
```

Duplicate Customer IDs: 0

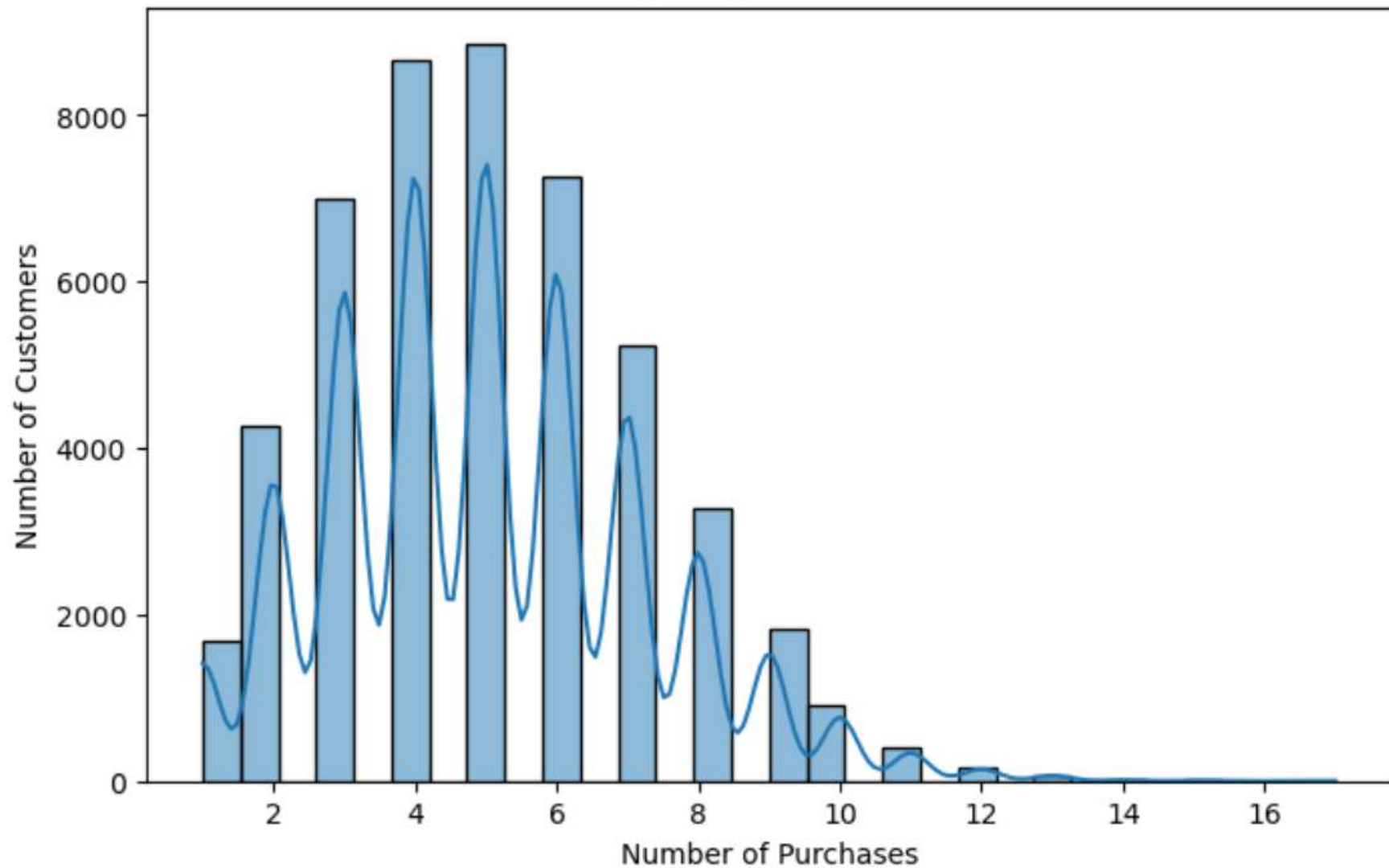
RFM Statistics:

	recency	frequency	monetary
count	49673.000000	49673.000000	49673.000000
mean	260.424979	5.032915	13716.559962
std	246.024001	2.206427	6840.208557
min	0.000000	1.000000	110.000000
25%	77.000000	3.000000	8719.000000
50%	184.000000	5.000000	13026.000000
75%	368.000000	6.000000	17966.000000
max	1352.000000	17.000000	55339.000000

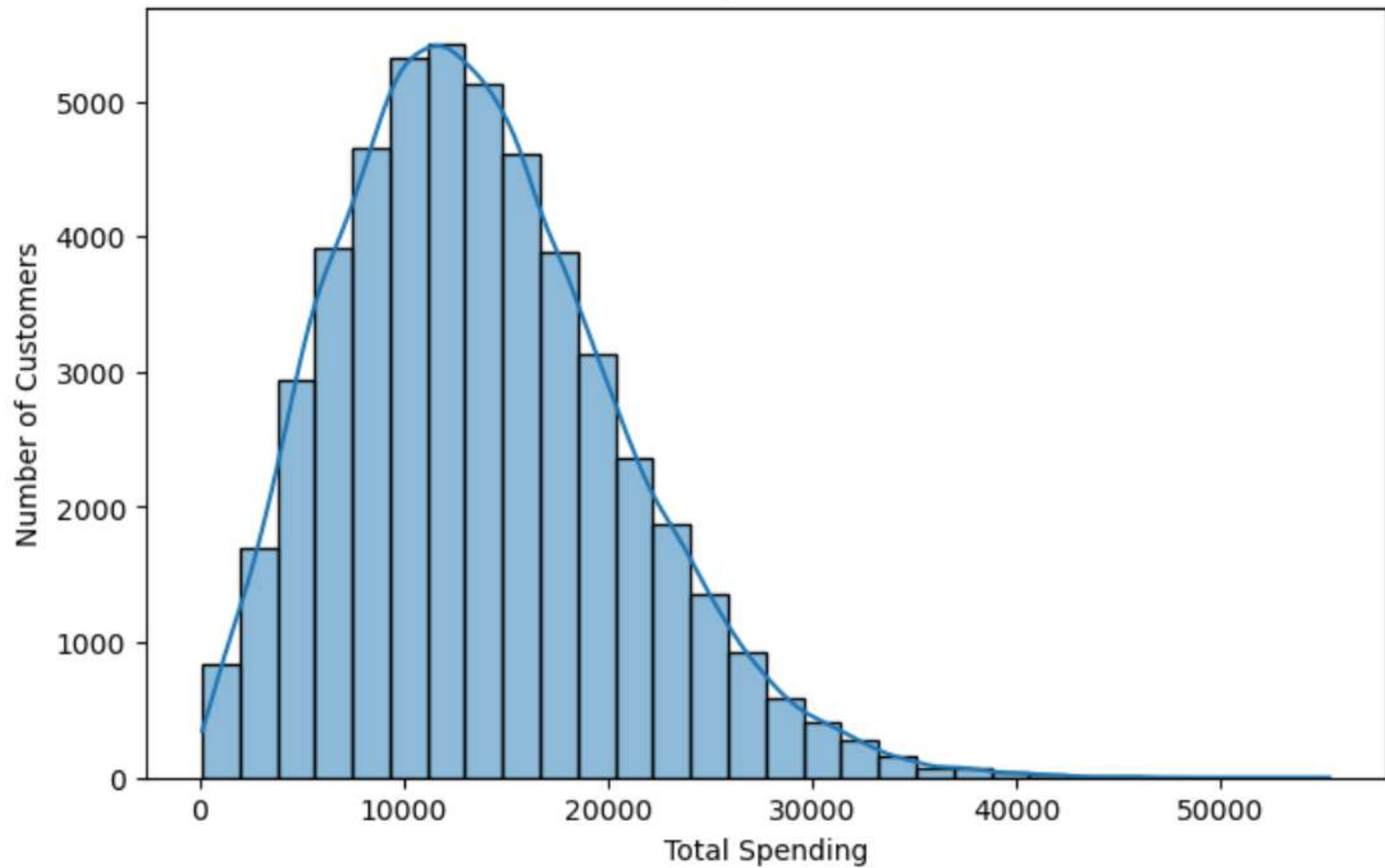
Recency Distribution



Frequency Distribution



Monetary Distribution



	customer_id	recency	frequency	monetary	R_Score	F_Score	M_Score	RFM_Score	RFM_Score_Code
0	1	57	1	3491	5	1	1	7	511
1	2	298	3	7988	2	1	2	5	212
2	3	88	8	22587	4	5	5	14	455
3	4	126	4	8715	4	2	2	8	422
4	5	170	8	12524	3	5	3	11	353
5	6	462	6	15517	1	4	4	9	144
6	7	565	3	7835	1	1	2	4	112
7	8	129	6	15265	4	4	4	12	444
8	9	126	5	14720	4	3	3	10	433
9	10	229	8	20857	3	5	5	13	355
10	11	136	10	23691	4	5	5	14	455
11	12	279	6	15378	2	4	4	10	244
12	13	88	4	7244	4	2	1	7	421
13	14	539	6	16161	1	4	4	9	144
14	15	102	5	19748	4	3	5	12	435
15	16	147	5	15246	3	3	4	10	334

16	17	13	9	24042	5	5	5	15	555
17	18	115	8	23726	4	5	5	14	455
18	19	125	5	8809	4	3	2	9	432
19	20	215	8	21428	3	5	5	13	355

R Score Distribution:

R_Score

1 9935

2 9934

3 9935

4 9934

5 9935

Name: count, dtype: int64

F Score Distribution:

F_Score

1 9935

2 9934

3 9935

4 9934

5 9935

Name: count, dtype: int64

M Score Distribution:

M_Score

1	9935
2	9934
3	9935
4	9934
5	9935

Name: count, dtype: int64

Customer Segment Counts:

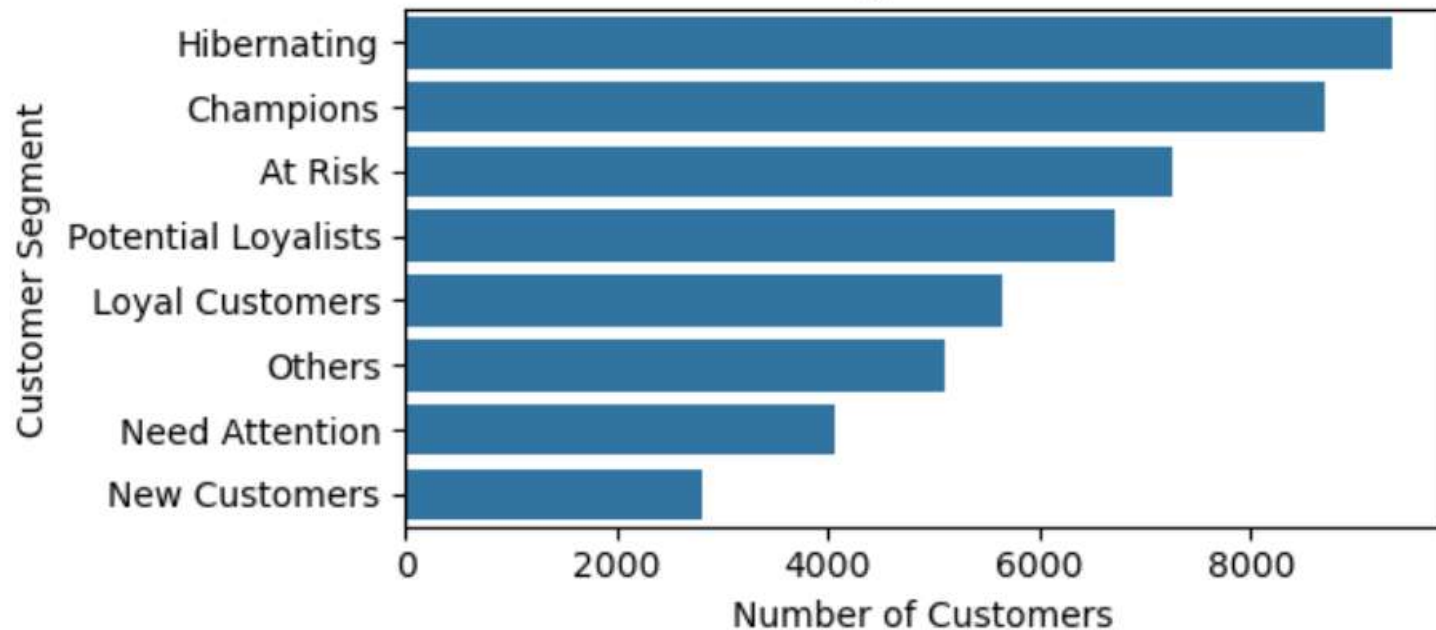
	Customer_Segment	Customer_Count
0	Hibernating	9347
1	Champions	8692
2	At Risk	7271
3	Potential Loyalists	6722
4	Loyal Customers	5644
5	Others	5111
6	Need Attention	4074
7	New Customers	2812

Customer Segment Distribution:

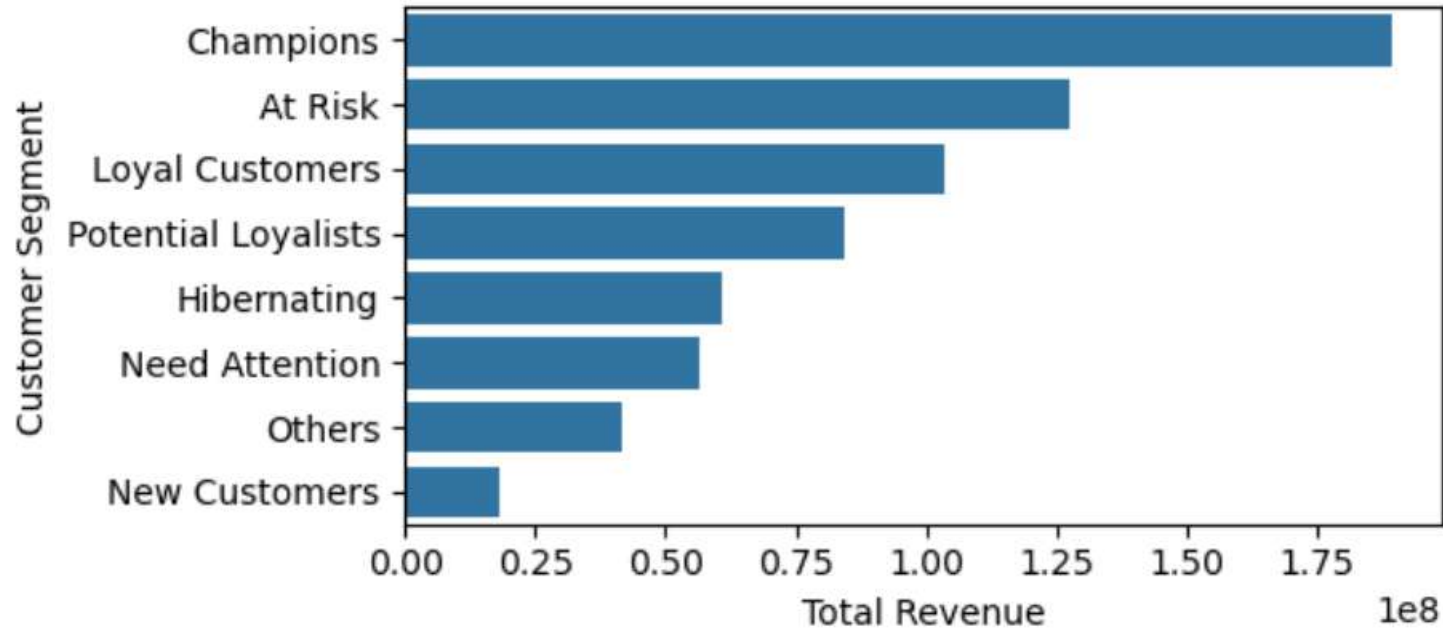
	Customer_Segment	Customer_Count	Customer_Percentage
0	Hibernating	9347	18.817064
1	Champions	8692	17.498440
2	At Risk	7271	14.637731
3	Potential Loyalists	6722	13.532503
4	Loyal Customers	5644	11.362310
5	Others	5111	10.289292
6	Need Attention	4074	8.201639
7	New Customers	2812	5.661023

RFM Segment Summary:									
	Customer_Segment	Customer_Count	Total_Revenue	Average_Spend	Average_Frequency	Average_Recency	Average_RFM_Score	Churn_Rate	Customer_Percentage
1	Champions	8692	189393382	21789.39	7.65	59.89	13.78	19.72	17.50
0	At Risk	7271	127579032	17546.28	6.16	404.24	9.54	19.20	14.64
3	Loyal Customers	5644	103289001	18300.67	6.97	153.39	11.83	20.15	11.36
7	Potential Loyalists	6722	84076914	12507.72	4.53	64.60	10.00	20.98	13.53
2	Hibernating	9347	60665965	6490.42	2.59	578.92	4.08	20.15	18.82
4	Need Attention	4074	56634408	13901.43	4.19	314.09	7.94	20.30	8.20
6	Others	5111	41409471	8102.03	3.91	254.12	6.60	20.04	10.29
5	New Customers	2812	18294510	6505.87	2.71	66.41	7.01	19.59	5.66

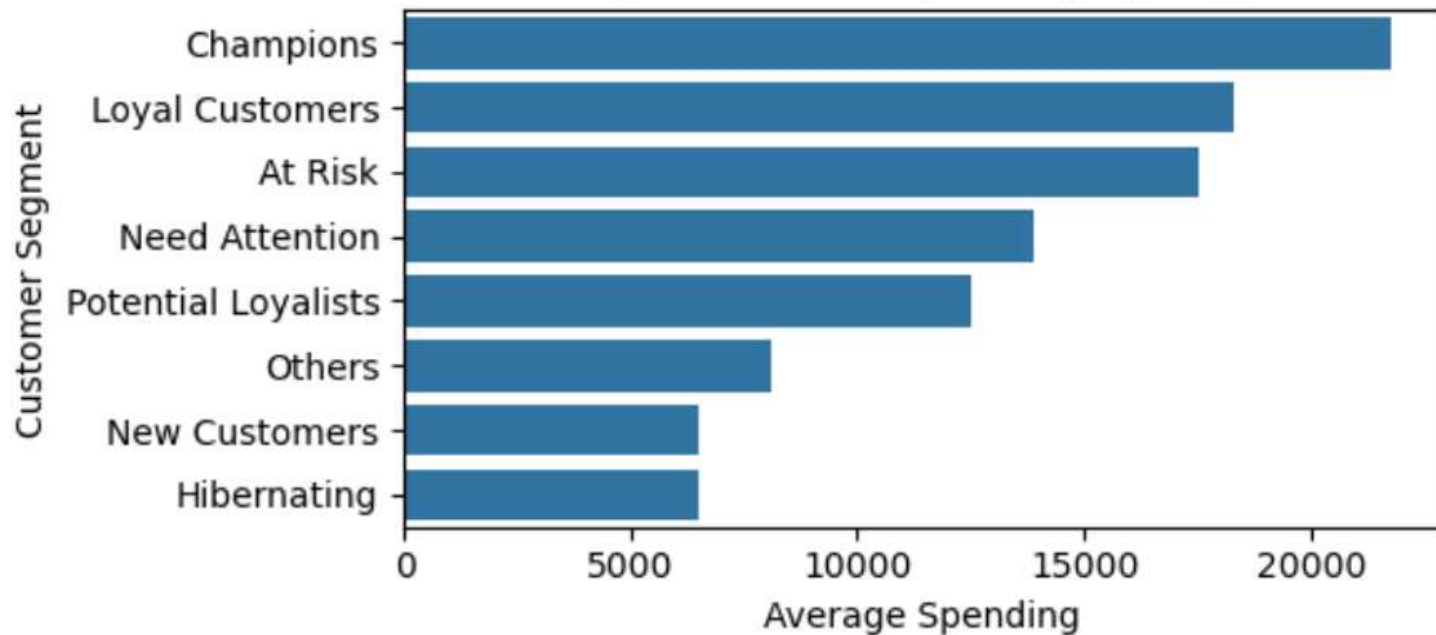
Customer Segment Distribution



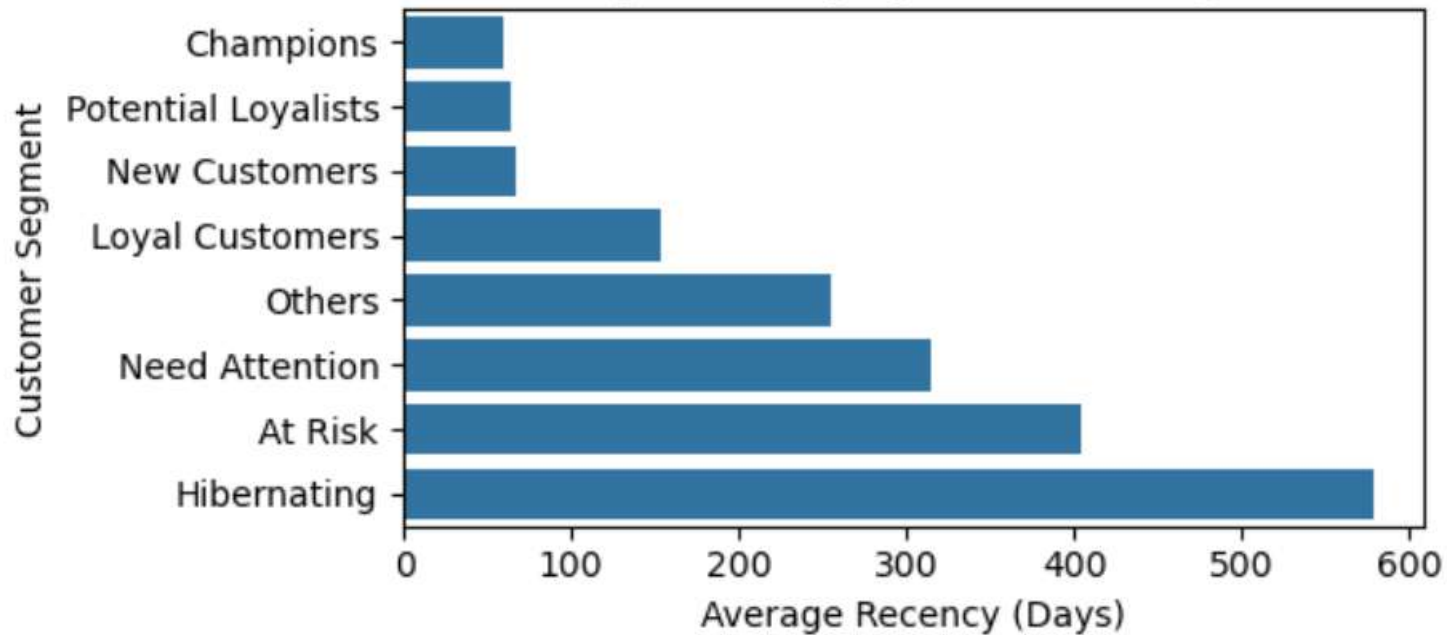
Total Revenue by Customer Segment



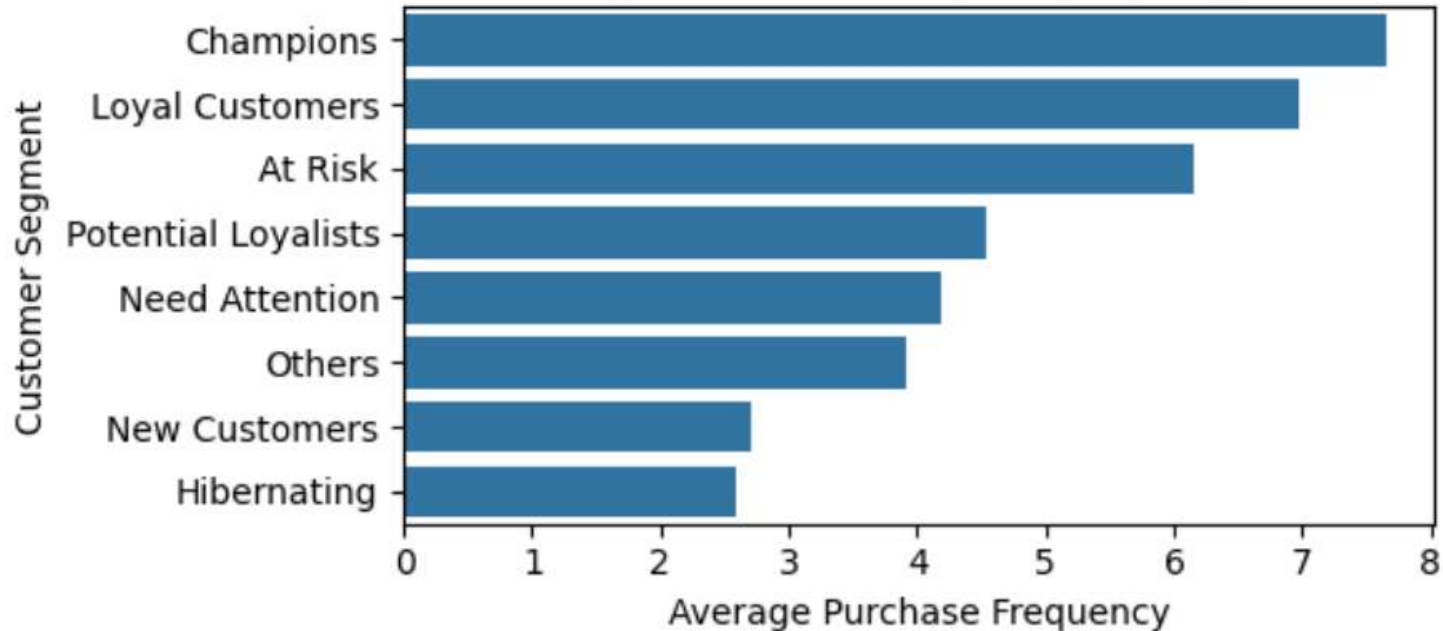
Average Customer Spending by Segment



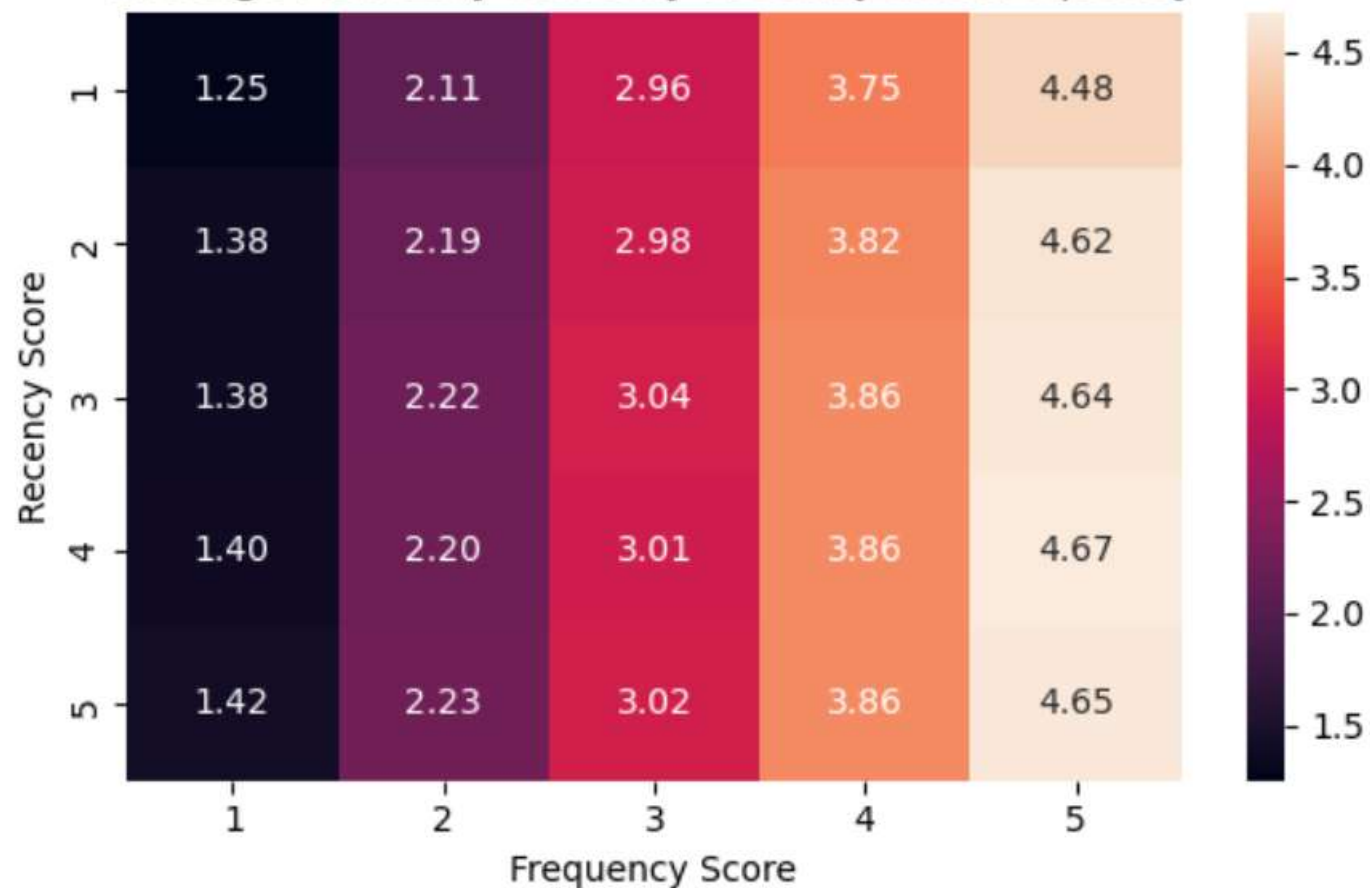
Average Recency by Customer Segment



Average Purchase Frequency by Segment



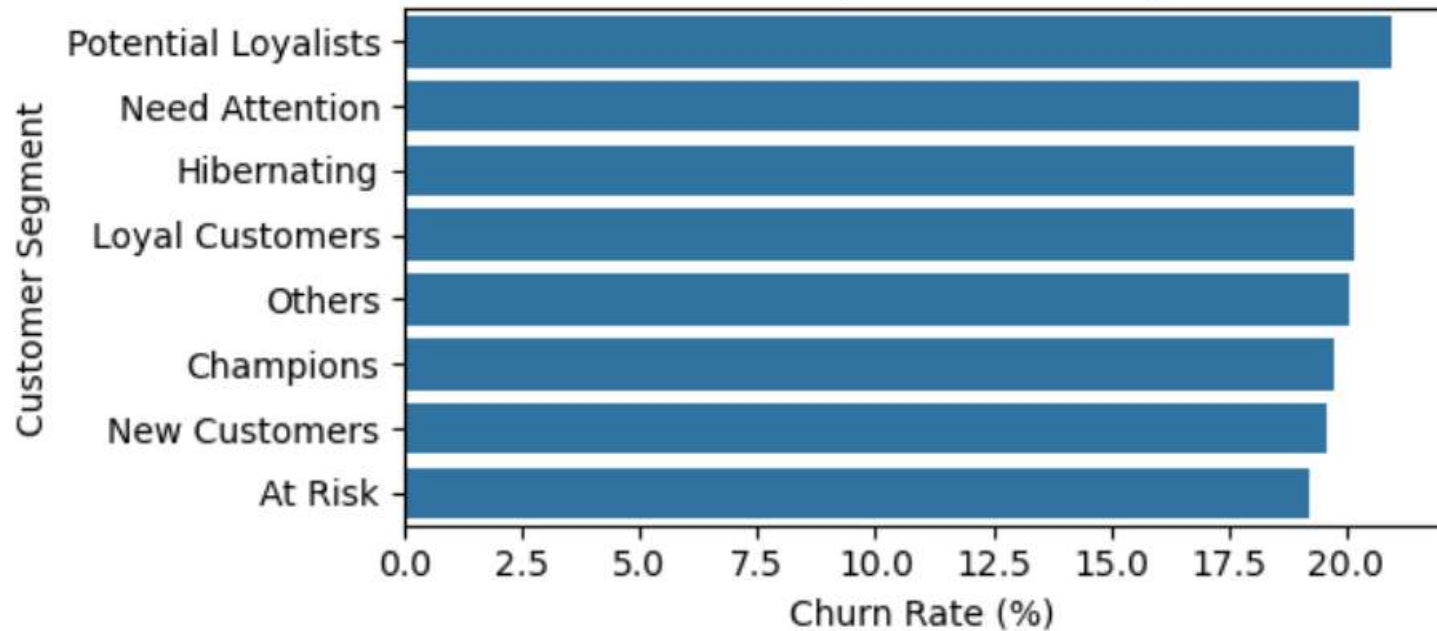
Average Monetary Score by Recency and Frequency



Churn by Customer Segment:

	Customer_Segment	Customers	Churned_Customers	Churn_Rate
7	Potential Loyalists	6722	1410	20.98
4	Need Attention	4074	827	20.30
2	Hibernating	9347	1883	20.15
3	Loyal Customers	5644	1137	20.15
6	Others	5111	1024	20.04
1	Champions	8692	1714	19.72
5	New Customers	2812	551	19.59
0	At Risk	7271	1396	19.20

Churn Rate by Customer Segment



Number of High-Value Customers: 13944

	customer_id	customer_name	recency	frequency	monetary	RFM_Score	Customer_Segment	churn
25847	25999	Briana Sanchez	26	8	20054	15	Champions	1
23637	23779	Kelly Lawrence	27	8	29150	15	Champions	0
23448	23588	Miranda Doyle	40	8	32689	15	Champions	0
23487	23627	Ryan West	9	10	31758	15	Champions	0
23518	23658	Wanda Charles	40	8	22096	15	Champions	0
23523	23663	Sean Schmidt	37	11	28017	15	Champions	0
23540	23681	John Strickland	38	7	23870	15	Champions	1
23544	23685	Kelli Lopez	32	7	20339	15	Champions	1
23570	23712	Tammy Moore	14	11	28375	15	Champions	0
23577	23719	Deanna Lawson	17	8	21859	15	Champions	0
23579	23721	Angela Ferrell	26	9	27161	15	Champions	0
23582	23724	Tammy Ochoa	36	10	26926	15	Champions	1
23604	23746	Christopher Hobbs	19	10	22840	15	Champions	1
23615	23757	Sarah Vance	42	7	22704	15	Champions	0

23615	23757	Sarah Vance	42	7	22704	15	Champions	0
23631	23773	Jesse Shepard	38	9	23463	15	Champions	0
23641	23784	Stephanie Skinner	34	8	23491	15	Champions	0
22835	22972	Tina Wright	28	9	22685	15	Champions	0
23675	23818	Michael Porter	31	8	21767	15	Champions	1
23680	23823	Christopher Black	18	11	33026	15	Champions	0
23687	23830	Brian Bond	47	7	19719	15	Champions	0

Number of At-Risk / Cannot-Lose Customers: 7271

	customer_id	customer_name	recency	frequency	monetary	RFM_Score	Customer_Segment	churn
27976	28144	Lindsay Morse	315	14	41347	12	At Risk	0
34827	35037	Mrs. Jennifer Lucas	349	11	41218	12	At Risk	1
48150	48471	David Duran	308	14	40436	12	At Risk	0
6742	6789	Andrew Salazar	294	11	39574	12	At Risk	0
35044	35256	Joseph Carey	291	11	39268	12	At Risk	1
31467	31659	Thomas Rosales	368	10	39068	12	At Risk	0
24833	24981	Samuel Lopez PhD	307	11	38511	12	At Risk	0

21149	21271	Michael Serrano	385	10	37985	12	At Risk	1
28918	29096	Amy Mitchell	430	13	37978	11	At Risk	0
29108	29286	Deborah Schwartz	282	11	37958	12	At Risk	0
28920	29098	Erin Romero	245	11	37770	12	At Risk	0
49210	49537	Nathan Torres	360	11	37631	12	At Risk	1
45686	45979	Kelly Smith	340	14	37593	12	At Risk	0
26174	26328	Eugene King	319	13	37519	12	At Risk	0
11186	11254	Alexis Hodges	385	11	37233	12	At Risk	0
40241	40504	David Dunn	446	11	37019	11	At Risk	0
32023	32218	Donna May	253	10	36837	12	At Risk	0
46555	46855	Tyler Foley	255	10	36804	12	At Risk	0
30830	31017	Sherry Phillips	256	11	36309	12	At Risk	0
34510	34720	Jeffrey Blanchard	534	11	36175	11	At Risk	0

High-Value Churned Customers: 4570

	customer_id	customer_name	recency	frequency	monetary	RFM_Score	Customer_Segment
46785	47087	Debbie Lopez	104	17	45700	14	Champions
9346	9405	Zachary Gaines	82	11	42479	14	Champions
6739	6786	Peter Williamson	14	12	42385	15	Champions
20720	20842	Elizabeth Yang	180	10	41501	13	Loyal Customers
20743	20865	Jacob Harrington	34	11	41391	15	Champions
34827	35037	Mrs. Jennifer Lucas	349	11	41218	12	At Risk
20039	20158	Curtis Allen	236	12	40306	13	Loyal Customers
49008	49335	Patricia Medina	231	13	40210	13	Loyal Customers
33222	33424	Howard Curtis	150	11	40152	13	Loyal Customers
18913	19030	Brianna Hanna	1	13	40034	15	Champions
2233	2247	Deborah Larsen	27	12	39314	15	Champions
35044	35256	Joseph Carey	291	11	39268	12	At Risk
38089	38329	Richard Mckinney	205	11	39101	13	Loyal Customers
31269	31459	Lance Nash	5	12	38847	15	Champions

26753	26912	Sandra Thompson	132	11	38523	14	Champions
21345	21469	Jaime Barton	1	12	38518	15	Champions
37918	38155	Julie Blackwell	10	11	38405	15	Champions
23990	24135	Lauren Lopez	28	13	38315	15	Champions
11130	11197	Shari Robbins	9	13	38162	15	Champions
28410	28582	Lawrence Ferguson	71	13	38123	14	Champions

Saved: customer_rfm_segments.csv

Saved: rfm_segment_summary.csv

Saved: high_value_customers.csv

Saved: at_risk_customers.csv

Saved: high_value_churned_customers.csv

=====

STEP 3 COMPLETED

=====

Total Customers: 49673

Average Recency: 260.42 days

Average Frequency: 5.03

Average Monetary Value: 13716.56

Average RFM Score: 9.0

Customer Segments:

Customer_Segment

Hibernating	9347
Champions	8692
At Risk	7271
Potential Loyalists	6722
Loyal Customers	5644
Others	5111
Need Attention	4074
New Customers	2812

Name: count, dtype: int64

Output files created:

1. customer_rfm_segments.csv
2. rfm_segment_summary.csv
3. high_value_customers.csv
4. at_risk_customers.csv
5. high_value_churned_customers.csv